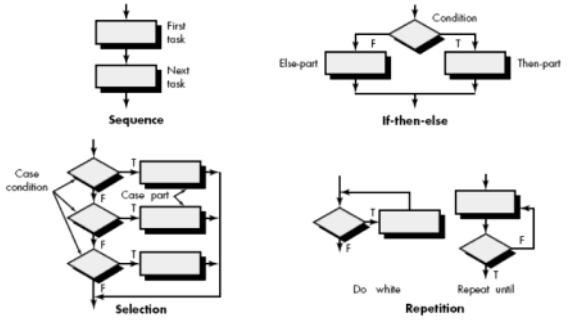
Yapısal Programlama

16 Mart 2026 Pazartesi 10:07

1970lerde dillere yapısal programlama desteği eklenmeye başlamıştır.
Yapısal programlama modülerliği de destekler.
Modülerlik ilk başlarda fonksiyonlarla sağlanırken günümüzde oop ile sağlanmaktadır.

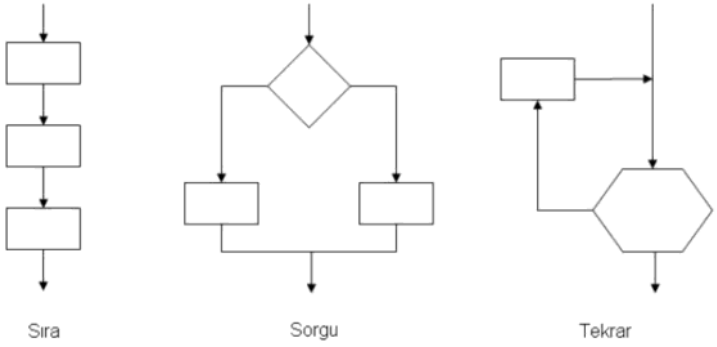
goto yerine do while benzeri döngüler kullanmayı teşvik eder.

Yapısal Programlamada Amaç:
- Programın kolay yazılması
- Okunabilir ve hataların kolay tespit edilmesi
- Bakım aşamasının iyileştirilmesi



Yapısal Programlamanın şartları
Yapısal programlamayı destekleyen bir dilde üç yapı tanımlanmış olmalıdır.
- Sıralı Yapılar
- Seçimlik Yapılar
- Yinelemeli Yapılar

do-while true olduğu sürece çalışır
repeat-until false olduğu sürece çalışır.
- ikisi de post-test döngüleridir. (gövde her türlü en az 1 kez çalışır)



Do-While x Repeat Until Döngüleri

```
i:Byte; Begin
i:=0;
repeat
i:=i+1;
writeln('Sakarya Üniversitesi');
Until i:=5;
Readln;
End.
```

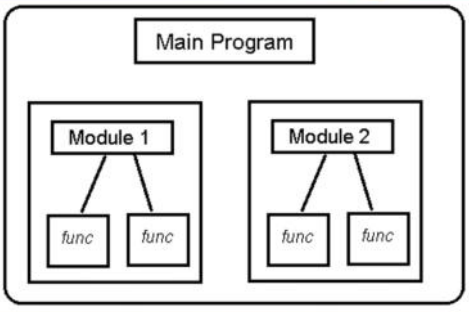
```
#include "stdio.h"
int main(){
        int i=0;
        do{
                i++;
                printf("%d ",i);
        }while(i != 5);
        return 0;
}
```

i 5'e eşit olana kadar çalışır

i, 5 değilken çalışır

Modül

- Küçük program parçacıkları
  - Sınıf
  - Fonksiyon
  - Blok



C gibi eski dillerde modülerlik bir fonksiyon hatta bir blok olarak anlaşılabilir.
Günümüzdeki dillerde sınıflar bu işi görür.
Javada modüller sınıflardır.

Modüllerin görevi bir sorumluluğu yerine getirmesidir. (single responsibilty)

c, c++ gibi dillerde bir şey döndürmeyen fonksiyonlara method denir.
Javada tamamen ndp olduğu için tüm fonksiyonlara method (üye fonk) denir.

Fonksiyonlar

16 Mart 2026 Pazartesi 10:36

Fonksiyonlarda Çağrı ve Dönüş

yığıt (stack veri yapısı)

Gerçek Parametre

Yerel Değişkenler	genelToplam
Yerel Değişkenler	List[4]
Yerel Değişkenler	List[3]
Yerel Değişkenler	List[2]
Yerel Değişkenler	List[1]
Yerel Değişkenler	List[0]
Formal parametre	Parca
Formal parametre	Toplam
Dinamik link	Dinamik kapsam bağlama için
Statik link	Statik kapsam bağlama için
Geri dönüş adresi	

Formal Parametre

aktivasyon kaydı.
fonksiyon kapatılınca yok edilirler

stack veri yapısı şeklinde işler.
son giren ilk çıkar
f1->f2->f3

en f3 çağrıldığından ilk f3 kapanır

içinde çağrılan fonk bittiğinde çağıran fonka dönmesi için ilk fonkun adresini kaydeder.

çağıran -> gerçek parametre

çağrılan(pasif) ->formel parametre

```
void fun1(int x) {
  int y;
  ...
  fun3(y);
  ... }
```

```
void fun2(double r) {
  int s,t;
  ...
  fun1(s);
  ... }
```

üstüne ekleniyor

	YIĞIT TEPESİ	
Fun2 için aktivasyon kaydı	Yerel değişkenler	t
	Yerel değişkenler	s
	Formal parametre	r
	Dinmik bağ (link)	
	Main'e dönüş	
Main için aktivasyon kaydı	Yerel değişkenler	p

	YIĞITIN TEPESİ	
Fun1 için aktivasyon kaydı	Yerel değişken	y
	Formal parametre	x
	Dinamik link	
	Fun2'ye dönüş	
Fun2 için aktivasyon kaydı	Yerel değişkenler	t
	Yerel değişkenler	s
	Formal parametre	r
	Dinmik bağ (link)	
	Main'e dönüş	
Main için aktivasyonkaydı	Yerel değişkenler	p

c c++'da fonksiyonun bir şey döndürüp döndürmediği kontrolü sıkıca yapılmaz bazı koşullar değer döndürmese bile hata vermez.

Özyinelemenin koşulları

1- Durma Noktası
2- Durma noktasına yaklaşması

- kodu hızlandırmaz kompleksliğini azaltır
- yineleme bağıntısı bulunmalıdır.

Parametreler

16 Mart 2026 Pazartesi 10:38

Parametre Aktarma Yöntemleri

- Değer ile aktarma
- Adres ile aktarma
- Referans ile aktarma
- Sonuç ile aktarma
- İsim ile aktarma

c ve java sadece değer ve adres ile aktarma destekler.
c++ referans ile aktarmayı da destekler

c# isim ile aktarma hariç hepsini destekler.

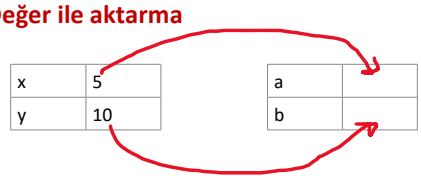
```
f(int a, int b){

}
```

```
int x = 5, y = 10;
f(x,y)
```

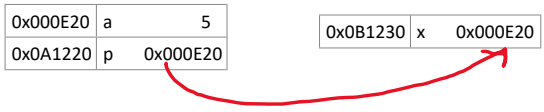
şeklinde çağrıldığında neyin func'a gideceğini belirleyen mantıktır.

Değer ile aktarma



- sadece değer bilgisi gönderilir.

Adres ile aktarma




```
#include "stdio.h"
void Degistir(int *x){
        *x = 100;
}
int main(){
        int a=10;
        int *p = &a;
        printf("a:%d\n",*p);
        Degistir(p);
        printf("a:%d\n",*p);
        return 0;
}
```

a'nın adrsini aktarma &a gönderiliyor

not: burda x'i değiştirirsek diğerlerine bağımlı olmaz. Çünkü p ve x hala değer ile aktarım mantığıyla çalışır a'nın adresini değer olarak alır

Referans ile aktarma:

```
//C# dilindeki kullanım
class Program
  {
    static void Degistir(ref int x,ref int y) {
      int z = x;
      x = y;
      y = z;
    }
    static void Main(string[] args) {
      int a = 100, b = 500;
      Degistir(ref a,ref b);
      Console.WriteLine("a:"+a);
      Console.WriteLine("b:" +b);
      Console.Read();
    }
  }
```

x ve y tamamen bir takma isimdir. formel parametre için yer ayrılmaz sadece doğrudan gerçek parametreler vardır.
dolayısıyla birebir aynı

```
void Ekle(in int x, in int y, ref int z) {
      ....
  }
```

c++daki mantık şöyle çalışır:
in int a --> const int &a
ref int a--> int &a

int* &a --> pointerın referans ile aktarılması

javada referans ile aktarım yoktur dolayısıyla gönderilen ve içerde değiştirilen her paremetre sonrasında = ile atanmalıdır. subNode = add(subNode); gibi

Sonuç ile aktarma:

```
static void GetUserInfo(out string name, out int age)
{
    name = "John Doe";
    age = 30;
}

static void Main()
{
    string userName;
    int userAge;


    GetUserInfo(out userName, out userAge);

    Console.WriteLine($"User Name: {userName}");
    Console.WriteLine($"User Age: {userAge}");
}
```

gönderilen dış parametrelere func sonunda değer atanmasını (bir nevi ekstra return değeri döndürmesini sağlar)

c++'ta benzetim yapılabilir.

- hem func yazılırken hem kullanılırken out parametresi kullanılmalı

bazen fonksiyonların birden fazla şey döndürmesi gerekebilir bu durumda sonuç ile çağırma yapılabilir.

İsim ile aktarma:

```
var notlar = [2.1,3.3,1.4,4,0.3];
function HarfleriHesapla(notDizisi,hesaplamaYontemi){
    var harfler=[];
    for(var i=0;i<notDizisi.length;i++){
        harfler.push(hesaplamaYontemi(notDizisi[i]));
    }
    return harfler;
}
function AFHesaplama(not){
    if(not < 1) return "FF";
    else if(not < 1.5) return "DD";
    else if(not < 2) return "DC";
    else if(not < 2.5) return "CC";
    else if(not < 3) return "CB";
    else if(not < 3.5) return "BB";
    else if(not < 4) return "BA";
    else return "AA";
}
function AHHesaplama(not){
    if(not < 1) return "H";
    else if(not < 1.5) return "G";
    else if(not < 2) return "F";
    else if(not < 2.5) return "E";
    else if(not < 3) return "D";
    else if(not < 3.5) return "C";
    else if(not < 4) return "B";
    else return "A";
}
console.log(HarfleriHesapla(notlar,AFHesaplama));
console.log(HarfleriHesapla(notlar,AHHesaplama));
```

doğrudan ismini alıp kullanmaktır

harfleriHesapla(notlar, Afhesapla());

c# gibi dillerde bu da yazılabilir fakat burada sadece func'tan dönen değer parametre olarak gönderilir aynı şey değildir

Varsayılan Parametre:

java desteklemez

```
def Asalmi(sayi,bolen=2):
        if sayi == bolen:
                return True;
        if sayi % bolen == 0:
                return False;
        return Asalmi(sayi,bolen+1)

sayi = input("Sayi:")
if Asalmi(sayi):
        print("Sayi asaldir.")
else:
        print("Sayi asal degildir.")
```

parametre gönderilmediğinde default olarak alacağı değeri belirlemeye yarar.

python:

```
def kuvvet(kutle=0,yercekim=9.8):
        return kutle*yercekim
```

```
# sonuç = sıfır
kuvvet()
```

```
# sonuç = 9.8
kuvvet(1)
```

```
# sonuç = 0
kuvvet(yercekim=10)
```

```
# sonuç = 50
kuvvet(5,10)
```

```
#sonuç = 40
kuvvet(yercekim=10,kutle=4)
```

c++ destekler

C#'ta Nullable değer:

```
void Add(int? x,int y){

}
```

Değişken Sayıda Parametre

```
public class Rastgele {
   public char Uret(char... karakterler){
      Random rnd = new Random();
      int rastgeleIndex = rnd.nextInt(karakterler.length);
      return karakterler[rastgeleIndex];
   }
}
```

dizi mantığıyla tutar.

```
public static void main(String[] args) {
      Rastgele r1 = new Rastgele();
      System.out.println(r1.Uret('a','b','c'));
      System.out.println(r1.Uret('a','f','g','u','y','z'));
      System.out.println(r1.Uret('a','k','e','x','b'));
      System.out.println(r1.Uret('v'));
   }
```

C

16 Mart 2026 Pazartesi 12:02

```c
#include <stdio.h>

//burada prototip tanımlanırsa hata vermez. //func'ın imzası
//direkt fonksiyon tanımı yukarıda da yapılabilir.
double f(double a, double b);

int main() {
    // printf("%lf\n", f(10.5,21.8));

    int x = 10.5, y = 21.8;
    printf("%lf\n", f(x, y)); //değer ile çağırma.

    return 0;
}

//fonksiyon tanımı altta olduğundan hata verir.
double f(double a, double b) {
    return a / b;
}
```

recursive:

```c
#include <stdio.h>
#include "stdarg.h"

typedef enum { false, true } boolean; //bool tipini tanımlıyoruz.

boolean asalmi(int parametreAdedi, ...){  //parametreAdedi, sayi, bolen
    va_list valist; //parametre tutan liste
    va_start(valist, parametreAdedi); //bu kadar parametre adedi valiste gönderildi
    int sayi = va_arg(valist, int); //sırada int türünden bir parametre gelicek onu al
    if(parametreAdedi == 1) { //ilk parametre harici 1 ise
        va_end(valist);
        return asalmi(2,sayi,2);
    }
    int bolen = va_arg(valist, int);
    va_end(valist);
    if(sayi == bolen) return true;
    if(sayi % bolen == 0) return false;
    return asalmi(2,sayi,bolen+1);
}

int main() {
    if(asalmi(1,13)) printf("Asal Sayi");
    else printf("Asal sayi değil");

    return 0;
}
```

Java

16 Mart 2026 Pazartesi 12:02

program.java:

```java
package hft06;

public class program {

    public static void degistir(Sayi s) {
        s.setDeger(400);
        System.out.println(s); //adresini bastırır.

        //şeklinde yapılırsa yeni bir nesne oluşturup onu s'ye atar. komple adresi değişir.
        //s = new Sayi(4000);
    }

    public static void main(String[] args) {
        Sayi sy = new Sayi(5912);

        short[] rakamlar = sy.rakamlar();
        for(short s : rakamlar) {
            System.out.println(s + " ");
        }


        System.out.println(sy.getDeger());
        degistir(sy); //adres ile aktarımdır. //dolaylı değişimdir.
        System.out.println(sy.getDeger());


        if(sy.asalMi()) System.out.println("\nSayı asaldır");
        else System.out.println("\nSayı asal değildir");
    }

}
```

```
5
9
1
2
5912
hft06.Sayi@33c7353a
400

Sayı asal değildir
```

sayi.java:

```java
public class Sayi {

    public int deger;
    private int uzunluk;

    public Sayi(int deger) {
        this.deger = deger;
        uzunluk = String.valueOf(deger).length();
    }

    public boolean asalMi(int... bolen) { //0'dan sonsuza kadar parametre sayısı yazılabilir
        if(bolen.length == 0) return asalMi(2); //baslangıc noktası
        if(deger == 1 || deger == bolen[0]) return true;
        if(deger % bolen[0] == 0) return false;
        return asalMi(bolen[0]+1);
    }

    public short[] rakamlar() {
        int tmp = deger;
        short[] basamaklar = new short[uzunluk];
        for(int index = uzunluk-1; tmp > 0; index--, tmp/=10) {
            basamaklar[index] = (short)(tmp%10);
        }
        return basamaklar;
    }

    public int getUzunluk() {
        return uzunluk;
    }

    public int getDeger() {
        return deger;
    }


    public int setDeger(int d) {
        return deger = d;
    }
}
```

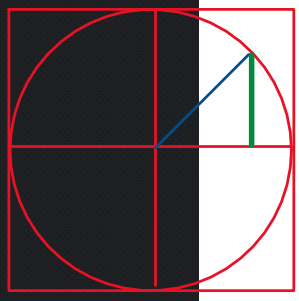
mat.java: (monte carlo algoritmasıyla pi sayısının elde edilmesi)

```java
package hft06prrr;

import java.util.Random;

public class Mat {

    public static double PIdeger(double denemeSayisi) {
        int basariliVurus = 0;
        Random rnd = new Random();
        for(double i = 0; i < denemeSayisi; i++) {
            double x = rnd.nextDouble();
            double y = rnd.nextDouble();
            double uzunluk = Math.sqrt(Math.pow(x,2) + Math.pow(y,2));
            if(uzunluk<1) basariliVurus++;
        }
        return 4*(basariliVurus/denemeSayisi);
    }
}
```



pisagor teoreminden:

uzunluk = √(x^2+y^2)
uzunluk < 1 şartı sağlayanlar çemberin içinde

```java
package hft06;

import hft06prrr.Mat;


public class program {

    public static void main(String[] args) {

        // PIdeger metodunu çağırıp sonucu değişkene atıyoruz

        double sonuc = Mat.PIdeger(1000000);
        System.out.println("Hesaplanan Pi Değeri: " + sonuc);

    }
}
```

```
Hesaplanan Pi Değeri: 3.141944
```